

# Reviewer Pack — Tree-Depth of Tseitin Graph Bounds Resolution Length

Entry 07532f1e404c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 02:37:50 UTC

## Tree-Depth of Tseitin Graph Bounds Resolution Length

**Entry ID:** 07532f1e404c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 02:37:50 UTC

### 1. Conjecture

**Field A** (mathematical branch): Geometric Group Theory **Field B** (complexity object): Resolution Proof Length

#### Statement:

For a Tseitin formula on a graph  $G$  with tree-depth  $\tau(G)$ , the minimal resolution refutation length is  $\geq 2^{\Omega(\tau(G))}$ . For graphs with  $\tau(G) \leq 2$  (e.g., planar graphs), the refutation length is polynomial.

#### Rationale (proposer's reasoning):

Tree-depth captures hierarchical complexity of graphs, which may mirror the recursive structure of resolution proofs. High tree-depth implies deep nested dependencies, requiring exponential proof steps. The geometric group theory perspective links graph embeddings to group actions, suggesting structural barriers to efficient refutations.

**Taxonomy category:** TSEITIN\_RES (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 6d04b877aecbf1be

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

default:  $\geq 80\%$  seeds must support, no counterexample

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.95       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.00       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.95       | SAFE             | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.95       | SAFE             | SAFE             |

| Barrier | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|---------|---------|------------|------------------|------------------|
|---------|---------|------------|------------------|------------------|

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from itertools import combinations, product

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda k: abs(A[k][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i, j, k in product(range(n), repeat=3):
        C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    n = len(A)
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        det += (-1)**j * A[0][j] * determinant([row[:j] + row[j+1:] for row in A[1:]])
    return det

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def is_planar(G):
    n = len(G)
```

```

if n <= 4:
    return True
for u in range(n):
    neighbors = [v for v in range(n) if G[u][v]]
    if len(neighbors) >= 5:
        subgraph = {u}
        for v in neighbors:
            subgraph.update(neighbors)
            if len(subgraph) > n - 3:
                return False
return True

def tree_depth(G):
    n = len(G)
    visited = [False] * n
    depth = [0] * n

    def dfs(u, d):
        visited[u] = True
        depth[u] = d
        for v in range(n):
            if G[u][v] and not visited[v]:
                dfs(v, d + 1)

    dfs(0, 0)
    return max(depth)

def tseitin_formula(G):
    n = len(G)
    literals = list(range(-n, 0)) + list(range(1, n+1))
    neg_literals = [-l for l in literals]
    clauses = []

    def add_clause(c):
        clauses.append(c)

    for u in range(n):
        if sum(G[u]) > 2:
            add_clause([literals[u]] + [neg_literals[v] for v in range(n) if G[u][v]])

    for u, v in combinations(range(n), 2):
        if G[u][v]:
            add_clause([-literals[u], literals[v]])
            add_clause([literals[u], -literals[v]])
            add_clause([-literals[v], literals[u]])
            add_clause([literals[v], -literals[u]])

    return literals, neg_literals, clauses

def dpll_with_clause_learning(G, literals, neg_literals, clauses):
    n = len(literals)
    assignment = {l: None for l in literals}
    learned_clauses = []

    def propagate():
        while True:

```

```

        found = False
        for c in clauses + learned_clauses:
            unit_clause = next((l for l in c if assignment.get(l) is None and assignment.get(l) is not False))
            if unit_clause:
                assignment[unit_clause] = True
                found = True
                break
        if not found:
            return True

def backtrack():
    while True:
        if all(assignment[l] is not None for l in literals):
            return False
        var = next(l for l in literals if assignment.get(l) is None)
        assignment[var] = False
        learned_clauses.append([var])

    if propagate() and backtrack():
        return True, assignment
    else:
        return False, {}

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    G = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]
    G = [row[:] for row in G]
    for i in range(n):
        G[i][i] = 0

    tau_G = tree_depth(G)
    literals, neg_literals, clauses = tseitin_formula(G)

    refutation_length = len(dpll_with_clause_learning(G, literals, neg_literals, clauses)[1])

    c = 0.3
    if tau_G > 2:
        expected_length = 2 ** (c * tau_G)
    else:
        expected_length = n**2

    conjecture_holds = refutation_length >= expected_length
    counterexample = "" if conjecture_holds else f"tau(G)={tau_G}, refutation_length={refutation_length}"

    return {
        "metric_name": "refutation_length",
        "metric_value": refutation_length,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
    results.append(result)

mean_length = sum(r["metric_value"] for r in results) / len(results)
std_length = math.sqrt(sum((r["metric_value"] - mean_length)**2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_length} std={std_length} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

tion_length=0"}
TRIAL: {"seed": 593, "metric_name": "refutation_length", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 631, "metric_name": "refutation_length", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 677, "metric_name": "refutation_length", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 727, "metric_name": "refutation_length", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 773, "metric_name": "refutation_length", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 821, "metric_name": "refutation_length", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 877, "metric_name": "refutation_length", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}
TRIAL: {"seed": 929, "metric_name": "refutation_length", "metric_value": 0, "instances_tested": 1, "conjecture_holds": true}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_19ccd50b.py", line 181, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_19ccd50b.py", line 181, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~
KeyError: 'seed'

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

Test crashed before producing valid results, preventing reliable assessment of conjecture support/falsification. | next: Fix test code to handle edge cases and re-run with proper error handling

## 11. Audit log (LLM calls)

**Total LLM calls:** 7

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | qwen3:8b         | 0         | 0   | 60850        |
| 2 | preregistration | ollama_remote | qwen3:8b         | 0         | 0   | 19823        |
| 3 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 16404        |
| 4 | novelty         | ollama_remote | qwen3:8b         | 0         | 0   | 11832        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 24941        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17228        |
| 7 | judge           | ollama_remote | qwen3:8b         | 0         | 0   | 13563        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 164641 ms total latency. Provider mix: {'ollama\_remote': 7}

*(full prompt+response transcripts available in research/audit/07532f1e404c.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/07532f1e404c.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/07532f1e404c.tar.gz (if generated)